

dynTools: Internal Tests

Ivan Jacob Agaloos Pesigan

Tests

```
#> test-dynTools-check-dyn-data
#> CRAN: tests skipped.
#> test-dynTools-combine-by-id-time
#> Test passed with 1 success.
#> Test passed with 1 success.
#> Test passed with 1 success.
#> Test passed with 1 success.
#> Test passed with 1 success.

#> test-dynTool-delete-initial-na-keep-index-dot
#> CRAN: tests skipped.
#> test-dynTools-delete-initial-na
#> CRAN: tests skipped.
#> test-dynTools-delete-observed-all-na
#> CRAN: tests skipped.
#> test-dynTools-delta-t-by-id
#> CRAN: tests skipped.
#> test-dynTools-detrend-by-id
#> CRAN: tests skipped.
#> test-dynTools-diagnostics-by-id
#> CRAN: tests skipped.
#> test-dynTools-drop-by-id
#> CRAN: tests skipped.
#> test-dynTools-dyn-utils-select-sort-dot
#> CRAN: tests skipped.
#> test-dynTools-elapsed-time-by-id
#> CRAN: tests skipped.
#> test-dynTools-filter-by-id
#> CRAN: tests skipped.
#> test-dynTools-filter-observed-rows
#> CRAN: tests skipped.
#> test-dynTools-flag-diagnostics-by-id
#> CRAN: tests skipped.
#> test-dynTools-get-drop-id
```

```

#> CRAN: tests skipped.
#> test-dynTools-get-observed-initial
#> CRAN: tests skipped.
#> test-dynTools-initial-na
#> CRAN: tests skipped.
#> test-dynTools-insert-na
#> CRAN: tests skipped.
#> test-dynTools-lag-by-id
#> CRAN: tests skipped.
#> test-dynTools-make-clock-time
#> CRAN: tests skipped.
#> test-dynTools-plot-by-id
#> CRAN: tests skipped.
#> test-dynTools-regularize-time-by-id
#> CRAN: tests skipped.
#> test-dynTools-replace-missing-code
#> CRAN: tests skipped.
#> test-dynTools-resolve-duplicate-id-time
#> CRAN: tests skipped.
#> test-dynTools-round-clock-time
#> CRAN: tests skipped.
#> test-dynTools-scale-by-id
#> CRAN: tests skipped.
#> test-dynTools-screen-by-id
#> CRAN: tests skipped.
#> test-dynTools-subset-by-id
#> CRAN: tests skipped.
#> test-dynTools-summary-by-id
#> CRAN: tests skipped.
#> test-dynTools-trim-initial-rows-by-id
#> CRAN: tests skipped.

#> [[1]]
#> [[1]] [[1]]
#> [[1]] [[1]]$value
#> [[1]] [[1]]$value[[1]]
#> NULL
#>
#>
#> [[1]] [[1]]$visible
#> [1] TRUE
#>
#>
#> [[1]] [[2]]
#> [[1]] [[2]]$value
#> [[1]] [[2]]$value[[1]]

```

```

#> [1] TRUE
#>
#>
#> [[1]][[2]]$visible
#> [1] TRUE
#>
#>
#> [[1]][[3]]
#> [[1]][[3]]$value
#> [[1]][[3]]$value[[1]]
#> NULL
#>
#>
#> [[1]][[3]]$visible
#> [1] TRUE
#>
#>
#> [[1]][[4]]
#> [[1]][[4]]$value
#> [[1]][[4]]$value[[1]]
#> NULL
#>
#>
#> [[1]][[4]]$visible
#> [1] TRUE
#>
#>
#> [[1]][[5]]
#> [[1]][[5]]$value
#> [[1]][[5]]$value[[1]]
#> NULL
#>
#>
#> [[1]][[5]]$visible
#> [1] TRUE
#>
#>
#> [[1]][[6]]
#> [[1]][[6]]$value
#> [[1]][[6]]$value[[1]]
#> NULL
#>
#>
#> [[1]][[6]]$visible
#> [1] TRUE

```

```

#>
#>
#> [[1]][[7]]
#> [[1]][[7]]$value
#> [[1]][[7]]$value[[1]]
#> NULL
#>
#>
#> [[1]][[7]]$visible
#> [1] TRUE
#>
#>
#> [[1]][[8]]
#> [[1]][[8]]$value
#> [[1]][[8]]$value[[1]]
#> NULL
#>
#>
#> [[1]][[8]]$visible
#> [1] TRUE
#>
#>
#> [[1]][[9]]
#> [[1]][[9]]$value
#> [[1]][[9]]$value[[1]]
#> NULL
#>
#>
#> [[1]][[9]]$visible
#> [1] TRUE
#>
#>
#> [[1]][[10]]
#> [[1]][[10]]$value
#> [[1]][[10]]$value[[1]]
#> NULL
#>
#>
#> [[1]][[10]]$visible
#> [1] TRUE
#>
#>
#> [[1]][[11]]
#> [[1]][[11]]$value
#> function (data, id, time, output = "time_ct", units = "hours",
#>   origin = c("by_id", "global"), input_units = NULL, scale = c("mean_dt",

```

```

#>     "median_dt"), scale_value = NULL, replace = FALSE)
#> {
#>   if (!is.data.frame(data)) {
#>     stop("`data` must be a data frame.", call. = FALSE)
#>   }
#>   if (anyDuplicated(names(data))) {
#>     stop("`data` must have unique column names.", call. = FALSE)
#>   }
#>   if (!is.character(id) || length(id) != 1L || is.na(id) ||
#>       id == "") {
#>     stop("`id` must be a non-empty character string.", call. = FALSE)
#>   }
#>   if (!is.character(time) || length(time) != 1L || is.na(time) ||
#>       time == "") {
#>     stop("`time` must be a non-empty character string.",
#>          call. = FALSE)
#>   }
#>   if (!is.character(output) || length(output) != 1L || is.na(output) ||
#>       output == "") {
#>     stop("`output` must be a non-empty character string.",
#>          call. = FALSE)
#>   }
#>   if (!is.logical(replace) || length(replace) != 1L || is.na(replace)) {
#>     stop("`replace` must be `TRUE` or `FALSE`.", call. = FALSE)
#>   }
#>   scale <- match.arg(scale)
#>   if (!is.null(scale_value)) {
#>     if (!is.numeric(scale_value) || length(scale_value) !=
#>         1L || is.na(scale_value) || !is.finite(scale_value) ||
#>         scale_value <= 0) {
#>       stop("`scale_value` must be `NULL` or a positive finite numeric scalar.",
#>            call. = FALSE)
#>     }
#>   }
#>   missing_vars <- setdiff(x = c(id, time), y = names(data))
#>   if (length(missing_vars) > 0L) {
#>     stop(paste0("The following variables are not in `data`: ",
#>                 paste(missing_vars, collapse = ", ", "."), call. = FALSE)
#>   }
#>   if (!replace && output %in% names(data)) {
#>     stop(paste0("`output` already exists in `data`: ", output,
#>                 "."), call. = FALSE)
#>   }
#>   temp_output <- ".elapsed_time_by_id_ct"
#>   while (temp_output %in% names(data)) {

```

```

#>     temp_output <- paste0(temp_output, ".")
#>   }
#>   data <- ElapsedTimeByID(data = data, id = id, time = time,
#>     output = temp_output, units = units, origin = origin,
#>     input_units = input_units, replace = FALSE)
#>   elapsed <- data[[temp_output]]
#>   if (nrow(data) == 0L) {
#>     if (replace) {
#>       data[[time]] <- numeric(0)
#>       time_variable <- time
#>     }
#>     else {
#>       data[[output]] <- numeric(0)
#>       time_variable <- output
#>     }
#>     data[[temp_output]] <- NULL
#>     attr(data, "time_ct_scale") <- list(variable = time_variable,
#>       original_units = units, scale = scale, scale_value = scale_value,
#>       mean_dt_original_units = NA_real_, median_dt_original_units = NA_real_,
#>       min_dt_original_units = NA_real_, max_dt_original_units = NA_real_,
#>       n_dt = 0L, n_positive_dt = 0L, interpretation = NA_character_)
#>     rownames(data) <- NULL
#>     return(data)
#>   }
#>   run <- rle(data[[id]])
#>   end <- cumsum(run$lengths)
#>   start <- end - run$lengths + 1L
#>   dt <- numeric(0)
#>   for (j in seq_along(start)) {
#>     index <- seq.int(from = start[j], to = end[j])
#>     elapsed_j <- elapsed[index]
#>     elapsed_j <- elapsed_j[!is.na(elapsed_j)]
#>     if (length(elapsed_j) < 2L) {
#>       next
#>     }
#>     dt <- c(dt, diff(elapsed_j))
#>   }
#>   dt <- dt[is.finite(dt)]
#>   dt_positive <- dt[dt > 0]
#>   if (length(dt_positive) == 0L) {
#>     stop("No positive consecutive time intervals were found.",
#>       call. = FALSE)
#>   }
#>   if (length(dt_positive) < length(dt)) {
#>     warning(paste("Non-positive consecutive time intervals",

```

```

#>           "were ignored when computing the CT time scale."),
#>           call. = FALSE)
#>   }
#>   if (is.null(scale_value)) {
#>       scale_value <- switch(scale, mean_dt = mean(dt_positive),
#>                             median_dt = median(dt_positive))
#>   }
#>   time_ct <- elapsed/scale_value
#>   if (replace) {
#>       data[[time]] <- time_ct
#>       time_variable <- time
#>   }
#>   else {
#>       data[[output]] <- time_ct
#>       time_variable <- output
#>   }
#>   data[[temp_output]] <- NULL
#>   attr(data, "time_ct_scale") <- list(variable = time_variable,
#>                                       original_units = units, scale = scale, scale_value = scale_value,
#>                                       mean_dt_original_units = mean(dt_positive), median_dt_original_units = median(dt_positive),
#>                                       min_dt_original_units = min(dt_positive), max_dt_original_units = max(dt_positive),
#>                                       n_dt = length(dt), n_positive_dt = length(dt_positive),
#>                                       interpretation = paste0("1 CT time unit = ", signif(scale_value,
#>                                                                 6), " ", units, "."))
#>   rownames(data) <- NULL
#>   data
#> }
#>
#> [[1]][[11]]$visible
#> [1] FALSE
#>
#>
#> [[1]][[12]]
#> [[1]][[12]]$value
#> [[1]][[12]]$value[[1]]
#> NULL
#>
#>
#> [[1]][[12]]$visible
#> [1] TRUE
#>
#>
#> [[1]][[13]]
#> [[1]][[13]]$value
#> [[1]][[13]]$value[[1]]

```

```
#> NULL
#>
#>
#> [[1]][[13]]$visible
#> [1] TRUE
#>
#>
#> [[1]][[14]]
#> [[1]][[14]]$value
#> [[1]][[14]]$value[[1]]
#> NULL
#>
#>
#> [[1]][[14]]$visible
#> [1] TRUE
#>
#>
#> [[1]][[15]]
#> [[1]][[15]]$value
#> [[1]][[15]]$value[[1]]
#> NULL
#>
#>
#> [[1]][[15]]$visible
#> [1] TRUE
#>
#>
#> [[1]][[16]]
#> [[1]][[16]]$value
#> [[1]][[16]]$value[[1]]
#> NULL
#>
#>
#> [[1]][[16]]$visible
#> [1] TRUE
#>
#>
#> [[1]][[17]]
#> [[1]][[17]]$value
#> [[1]][[17]]$value[[1]]
#> NULL
#>
#>
#> [[1]][[17]]$visible
#> [1] TRUE
#>
```



```

#>
#> [[1]] [[18]]
#> [[1]] [[18]]$value
#> [[1]] [[18]]$value[[1]]
#> NULL
#>
#>
#> [[1]] [[18]]$visible
#> [1] TRUE
#>
#>
#> [[1]] [[19]]
#> [[1]] [[19]]$value
#> [[1]] [[19]]$value[[1]]
#> NULL
#>
#>
#> [[1]] [[19]]$visible
#> [1] TRUE
#>
#>
#> [[1]] [[20]]
#> [[1]] [[20]]$value
#> [[1]] [[20]]$value[[1]]
#> NULL
#>
#>
#> [[1]] [[20]]$visible
#> [1] TRUE
#>
#>
#> [[1]] [[21]]
#> [[1]] [[21]]$value
#> [[1]] [[21]]$value[[1]]
#> NULL
#>
#>
#> [[1]] [[21]]$visible
#> [1] TRUE
#>
#>
#> [[1]] [[22]]
#> [[1]] [[22]]$value
#> [[1]] [[22]]$value[[1]]
#> NULL
#>

```

```
#>
#> [[1]][[22]]$visible
#> [1] TRUE
#>
#>
#> [[1]][[23]]
#> [[1]][[23]]$value
#> [[1]][[23]]$value[[1]]
#> NULL
#>
#>
#> [[1]][[23]]$visible
#> [1] TRUE
#>
#>
#> [[1]][[24]]
#> [[1]][[24]]$value
#> [[1]][[24]]$value[[1]]
#> NULL
#>
#>
#> [[1]][[24]]$visible
#> [1] TRUE
#>
#>
#> [[1]][[25]]
#> [[1]][[25]]$value
#> [[1]][[25]]$value[[1]]
#> NULL
#>
#>
#> [[1]][[25]]$visible
#> [1] TRUE
#>
#>
#> [[1]][[26]]
#> [[1]][[26]]$value
#> [[1]][[26]]$value[[1]]
#> NULL
#>
#>
#> [[1]][[26]]$visible
#> [1] TRUE
#>
#>
#> [[1]][[27]]
```

```

#> [[1]][[27]]$value
#> [[1]][[27]]$value[[1]]
#> NULL
#>
#>
#> [[1]][[27]]$visible
#> [1] TRUE
#>
#>
#> [[1]][[28]]
#> [[1]][[28]]$value
#> [[1]][[28]]$value[[1]]
#> NULL
#>
#>
#> [[1]][[28]]$visible
#> [1] TRUE
#>
#>
#> [[1]][[29]]
#> [[1]][[29]]$value
#> [[1]][[29]]$value[[1]]
#> NULL
#>
#>
#> [[1]][[29]]$visible
#> [1] TRUE
#>
#>
#> [[1]][[30]]
#> [[1]][[30]]$value
#> [[1]][[30]]$value[[1]]
#> NULL
#>
#>
#> [[1]][[30]]$visible
#> [1] TRUE
#>
#>
#> [[1]][[31]]
#> [[1]][[31]]$value
#> [[1]][[31]]$value[[1]]
#> NULL
#>
#>

```

```
#> [[1]][[31]]$visible  
#> [1] TRUE
```

Environment

```
ls()  
#> [1] "root"
```

Class

```
#> [[1]]  
#> [1] "root_criterion"
```

References

R Core Team. (2026). *R: A language and environment for statistical computing*. R Foundation for Statistical Computing. Vienna, Austria. <https://www.R-project.org/>